ROB 5014 Math and Simulation for Robotics
Lecture: Geometric Calibration

Hyungpil Moon, Professor

SUNG KYUN KWAN
UNIVERSITY (SKKU)

Department of Mechanical Engineering
Department of Intelligent Robotics

Contents

- Least squares
  - Choleski decomposition (LU) LL^T
  - QR factorization
- Robot manipulator Kinematic (geometric) Calibration
- Camera geometric calibration

Problem of Least Squares

- Variable β has a linear dependance on $\alpha_1, \alpha_2, \cdots, \alpha_n$
  - $\beta = x_1\alpha_1 + x_2\alpha_2 + \cdots + x_n\alpha_n$
- We have m observations, then $Ax = b, A \in \Re^{m\times n}, m > n$

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \ldots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \ldots + a_{2n}x_n = b_2 \\ \ldots \\ a_{m1}x_1 + a_{m2}x_2 + \ldots + a_{mn}x_n = b_m \end{cases}$$

- Since equality is usually not exactly satisfied when m〉n, the least square solution x minimizes the residual $r(x) = b - Ax$.
  - $\min||r(x)||_2^2 = \min||b - Ax||_2^2$

Solution Methods

- Normal equations method
  - Cholesky factorization
- Orthogonal method: QR factorization
- Singular Value Decomposition (SVD)

Choleski's Decomposition: $A = LL^T$ (1/2)

- Conditions
  - A must be symmetric
  - A must be positive definite
- Example: a 3x3 matrix

$$\begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \\ A_{31} & A_{32} & A_{33} \end{bmatrix} = \begin{bmatrix} L_{11} & 0 & 0 \\ L_{21} & L_{22} & 0 \\ L_{31} & L_{32} & L_{33} \end{bmatrix} \begin{bmatrix} L_{11} & L_{21} & L_{31} \\ 0 & L_{22} & L_{32} \\ 0 & 0 & L_{33} \end{bmatrix} = \begin{bmatrix} L_{11}^2 & L_{11}L_{21} & L_{11}L_{31} \\ L_{11}L_{21} & L_{21}^2 + L_{22}^2 & L_{21}L_{31} + L_{22}L_{32} \\ L_{11}L_{31} & L_{21}L_{31} + L_{22}L_{32} & L_{31}^2 + L_{32}^2 + L_{33}^2 \end{bmatrix}$$

- Solution
  - $A_{11} = L_{11}^2 \rightarrow L_{11} = \sqrt{A_{11}}$, $A_{21} = L_{11}L_{21} \rightarrow L_{21} = A_{21}/L_{11}$, $A_{31} = L_{11}L_{31} \rightarrow L_{31} = A_{31}/L_{11}$
  - $A_{22} = L_{21}^2 + L_{22}^2 \rightarrow L_{22} = \sqrt{A_{22} - L_{21}^2}$, $A_{32} = L_{21}L_{31} + L_{22}L_{32} \rightarrow L_{32} = (A_{32} - L_{21}L_{31})/L_{22}$
  - $A_{33} = L_{31}^2 + L_{32}^2 + L_{33}^2 \rightarrow L_{33} = \surd(A_{33} - L_{31}^2 - L_{32}^2)$

Choleski's Decomposition: $A = LL^T$ (2/2)

- Generalization
  - $(LL^T)_{ij} = L_{i1}L_{j1} + L_{i2}L_{j2} + \cdots L_{ij}L_{jj} = \Sigma_k^j L_{ik}L_{jk}, i \geq j$
  - $A_{ij} = \Sigma_{k=1}^{j} L_{ik}L_{jk}, i = j, j+1, \cdots, n, \quad j = 1,2,\cdots,n$
  - Or $A_{ij} = L_{ij}L_{jj} + \Sigma_{k=1}^{j-1} L_{ik}L_{jk}$
  - From a diagonal term, $L_{jj} = \sqrt{}(A_{jj} - \Sigma_{k=1}^{j-1} L_{jk}^2)$ (for $j = 2,3,\cdots,n$)
  - For nondiagonal term, $Lij = \frac{A_{ij} - \Sigma_{k=1}^{j-1} L_{ik}L_{jk}}{L_{jj}}$ (for $j = 2,3,\cdots,n-1, \qquad i = j+1, j+2, \cdots, n$)

Normal Equation Method

- Normal Equation: $A^T A x = A^T b$
- If A has full column rank, then
  - $A^T A$ is symmetric
  - $A^T A$ is positive definite
  - $A \in \Re^{m \times \mathrm{n}}$, rank(A)=n
  - $A^T A = LL^T$ where $L \in \Re^{n \times n}$ lower triangular matrix
- Algorithm
  - For A, define a submatrix $A_{i:i',j:j'}$ to be $(i' - i + 1) \times (j' - j + 1)$ submatrix of A with upper left corner a_{ij} and the lower right corner $a_{i'j'}$.
  - Let $R = A$
  - For $k = 1$ to m
    - For $j = k + 1$ to m
      - $R_{j:j:m} = R_{j,j:m} - R_{k,j:m} R_{kj} / R_{kk}$
    - $R_{k,k:m} = R_{k,k:m} / \sqrt{R_{kk}}$

Orthogonal Method: QR Factorization (1/3)

- \For $A \in \Re^{m\times n}$, let $A = [a_1 \;\; a_2 \;\; \cdots a_n]$.
- We wish to produce a sequence of orthonormal vectors $q_1, q_2, \cdots$ spanning the same column space of A.
- Or equivalently, $A = QR$,
  - $Q \in \Re^{m\times n}$ has orthonormal columns, $Q^T Q = I$
  - $R \in \Re^{n\times n}$ is upper triangular
- Least squares problem is equivalent to the solution of $Rx = Q^T b$, where $A = QR$
  - Proof:
  - $A = Q\begin{bmatrix} R \\ 0 \end{bmatrix}$ where $Q \in \Re^{m\times m}$: orthogonal
  - $\rightarrow Q^T\left(Ax = Q\begin{bmatrix} R \\ 0 \end{bmatrix} x = b\right) \rightarrow Q^T Ax = \begin{bmatrix} R \\ 0 \end{bmatrix} x = \begin{bmatrix} \hat{b}_1 \\ \hat{b}_2 \end{bmatrix} = Q^T b$
  - If we partition $Q = [Q_1 \; Q_2]$ where $Q_1 \in \Re^{m\times n}$
  - Then, $A = Q\begin{bmatrix} R \\ 0 \end{bmatrix} = [Q_1 \; Q_2]\begin{bmatrix} R \\ 0 \end{bmatrix} = Q_1 R$
  - Therefore, the solution to the least squares problem $Ax = b$ is the solution to $Q_1^T Ax = Rx = \hat{b}_1 = Q_1^T b$

Orthogonal Method: QR Factorization (2/3)

- Householder Transformations
- Main idea
  - Multiply the matrix A by a sequence of simple orthogonal matrices Q_k to shape A into a upper triangular matrix.
  - After n−times, we get $Q^T = Q_n \cdots Q_2 Q_1 A = R$
  - To construct Q^T, we choose $Q_k = \begin{bmatrix} I & 0 \\ 0 & F \end{bmatrix}$ where $F = I - 2\frac{vv^T}{v^T v}, v = ||x||e_1 - x$
  - This achieves $Fx = \begin{bmatrix} ||x|| \\ 0 \\ \vdots \\ 0 \end{bmatrix} = ||x||e_1$

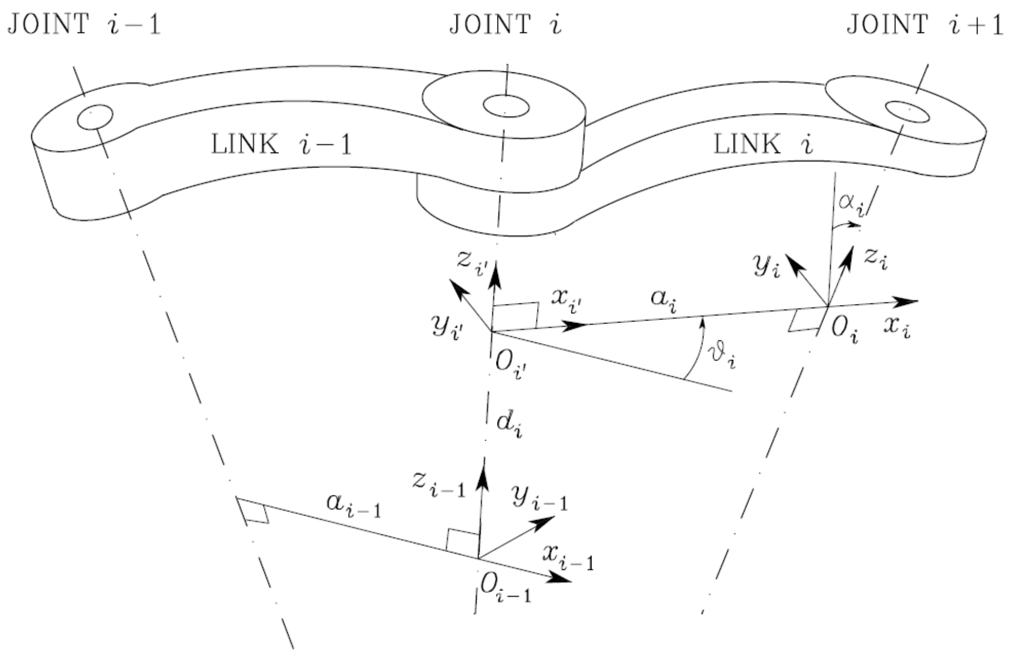
Orthogonal Method: QR Factorization (2/3)

- Algorithm
- For $k = 1$ to n
  - $v = A_{k:m,k}$
  - $u_k = v - ||v||e_1$
  - $u_k = \frac{u_k}{||u_k||}$
  - $A_{k:m,k:n} = A_{k:m,k:n} - 2u_k(u_k^T A_{k:m,k:n})$

- Note that the last line: $FA = \left(I - 2\frac{vv^T}{v^T v}\right)A = A - 2\frac{v(v^T A)}{v^T v}$

Kinematic Calibration in DH parameters (1/2)

- DH convention
  - Link length a_i : distance between O_i and $O_{i'}$
  - Offset d_i : distance from O_{i-1} to $O_{i'}$ along z_{i-1}
  - Twist angle α_i : angle between z's about x_i
  - Joint angle θ_i : angle between $x's$ about z_{i-1}
  - $T_i^{i-1}(q_i) = \begin{bmatrix} c_{\theta_i} & -s_{\theta_i}c_{\alpha_i} & s_{\theta_i}s_{\alpha_i} & a_i c_{\theta_i} \\ s_{\theta_i} & c_{\theta_i}c_{\alpha_i} & -c_{\theta_i}s_{\alpha_i} & a_i s_{\theta_i} \\ 0 & s_{\alpha_i} & c_{\alpha_i} & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$



- Forward Kinematics in DH convention
  - $T_n^0(\boldsymbol{q}) = T_1^0(q_1)T_2^1(q_2)\cdots T_n^{n-1}(q_n)$
  - $\boldsymbol{p} = \boldsymbol{f}(\boldsymbol{a}, \boldsymbol{\alpha}, \boldsymbol{d}, \boldsymbol{\theta})$
  - $\boldsymbol{a} = [a_1 \cdots a_n]^T, \boldsymbol{\alpha} = [\alpha_1 \cdots \alpha_n]^T, \boldsymbol{d} = [d_1 \cdots d_n]^T, \boldsymbol{\theta} = [\theta_1 \cdots \theta_n]^T$
- Due to many causes, the real robot hardware end pose is different from FK.

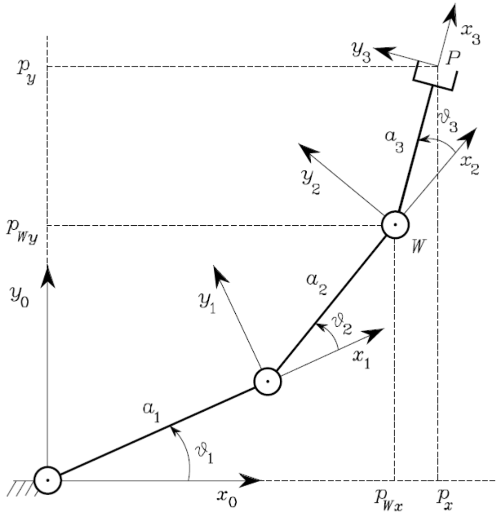
Kinematic Calibration in DH parameters (2/2)

- Linearization
  - Normally we use only position of the end effector
  - Measured pose: $\boldsymbol{p}_m$
  - Nominal pose (from DH): $\boldsymbol{p}_n$
  - Error model: $\Delta\boldsymbol{p} = \boldsymbol{p}_m - \boldsymbol{p}_n = \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{a}}\Delta\boldsymbol{a} + \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{\alpha}}\Delta\boldsymbol{\alpha} + \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{d}}\Delta\boldsymbol{d} + \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{\theta}}\Delta\boldsymbol{\theta}$
  - Augmented parameter: $\boldsymbol{x} = [\boldsymbol{a}^T \ \boldsymbol{\alpha}^T \ \boldsymbol{d}^T \ \boldsymbol{\theta}^T]^T, \Delta\boldsymbol{x} = \boldsymbol{x}_m - \boldsymbol{x}_n$
  - Jacobian (kinematic calibration) matrix: $\boldsymbol{\Phi} = \begin{bmatrix} \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{a}} & \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{\alpha}} & \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{d}} & \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{\theta}} \end{bmatrix} \in \Re^{m \times 4n}$
- Calibration problem: $\Delta\boldsymbol{p} = \boldsymbol{\Phi}(\boldsymbol{x}_n)\Delta\boldsymbol{x}$
- We need measurements of l poses
  - $\Delta\bar{\boldsymbol{p}} = \begin{bmatrix} \Delta\boldsymbol{p}_1 \\ \vdots \\ \Delta\boldsymbol{p}_l \end{bmatrix} = \begin{bmatrix} \boldsymbol{\Phi}_1 \\ \vdots \\ \boldsymbol{\Phi}_l \end{bmatrix}\Delta\boldsymbol{x} = \bar{\boldsymbol{\Phi}}\Delta\boldsymbol{x}$
- Solution: Pseudo inverse: $\Delta\boldsymbol{x} = (\bar{\boldsymbol{\Phi}}^T\bar{\boldsymbol{\Phi}})^{-1}\bar{\boldsymbol{\Phi}}^T\Delta\bar{\boldsymbol{p}}$
- New DH parameters: $\boldsymbol{x}' = \boldsymbol{x}_n + \Delta\boldsymbol{x}$

Simple Example (1/2)

- Position

$$x = L_1 \cos\theta_1 + L_2 \cos(\theta_1 + \theta_2) + L_3 \cos(\theta_1 + \theta_2 + \theta_3),$$
$$y = L_1 \sin\theta_1 + L_2 \sin(\theta_1 + \theta_2) + L_3 \sin(\theta_1 + \theta_2 + \theta_3),$$
$$\phi = \theta_1 + \theta_2 + \theta_3.$$



- Forward Kinematics
  - $T_{03} = T_{01} T_{12} T_{23}$

Nominal values

Link	a_i	α_i	d_i	ϑ_i
1	a_1	0	0	ϑ_1
2	a_2	0	0	ϑ_2
3	a_3	0	0	ϑ_3

Simple Example (2/2)

- Position

$$
\begin{aligned}
x &= L_1 \cos\theta_1 + L_2 \cos(\theta_1 + \theta_2) + L_3 \cos(\theta_1 + \theta_2 + \theta_3), \\
y &= L_1 \sin\theta_1 + L_2 \sin(\theta_1 + \theta_2) + L_3 \sin(\theta_1 + \theta_2 + \theta_3), \\
\phi &= \theta_1 + \theta_2 + \theta_3.
\end{aligned}
$$

- We need to use a general form of DH convention
  - $T_3^0 = T_{01} T_{12} T_{23}$
- Find an error model as discussed earlier.
- This is quite complicated, cumbersome, troublesome, but inevitable if we use DH convention.
  - So, use a symbolic computation of your favorite programming
- More importantly, DH does not provide all the degree of freedom of coordinate transform between neighboring frames.
  - We use 6-parameter forward kinematics model:

Back to Camera Calibration

- Previously, we have discussed that the camera calibration is a homogeneous problem.

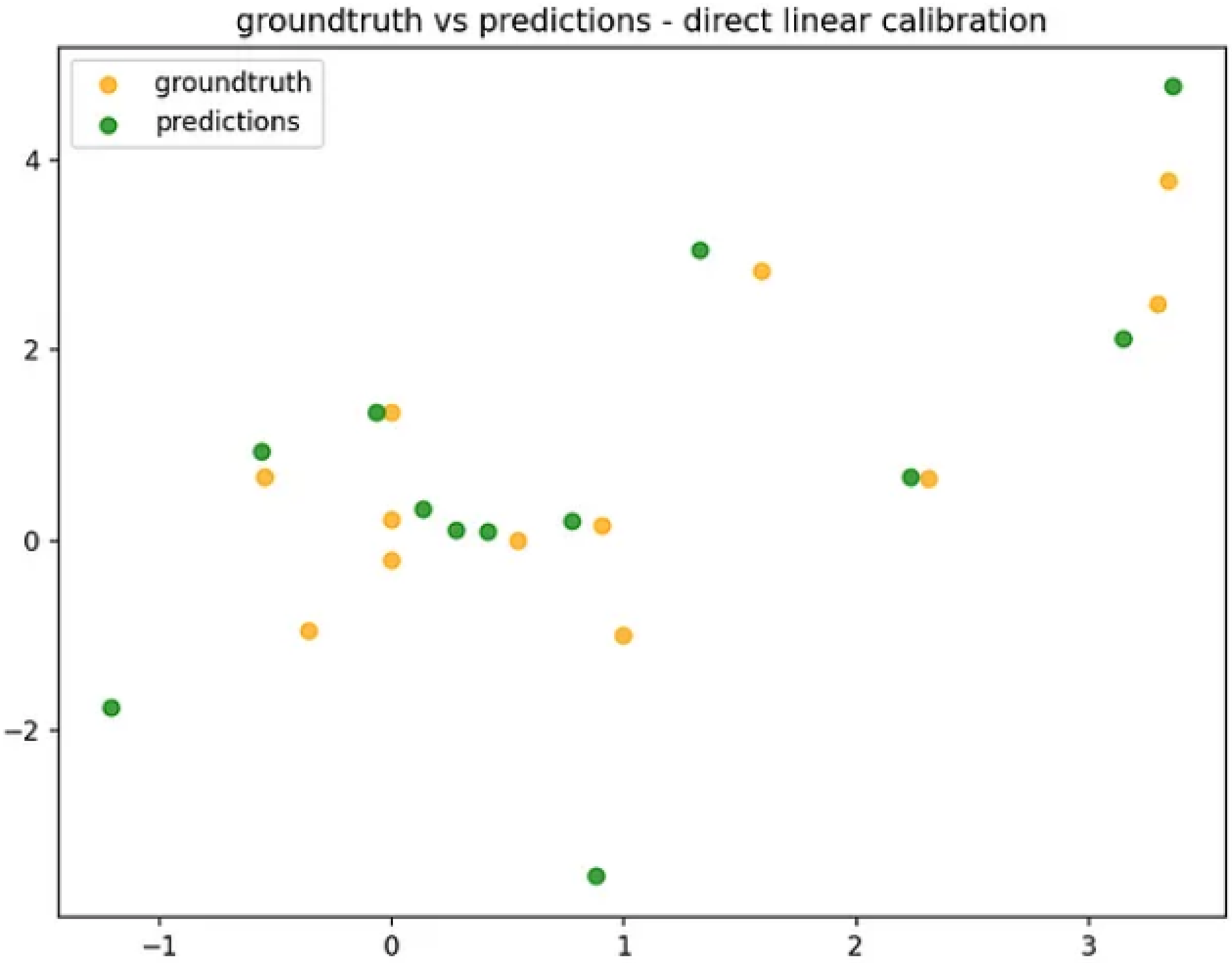
8-point algorithm: Ax=0
(Homogeneous equation)

Solution: x is the last column of V, where $A = UDV^T$ is the SVD of A

$$\begin{bmatrix} u_1u_1' & v_1u_1' & u_1' & u_1v_1' & v_1v_1' & v_1' & u_1 & v_1 & 1 \\ u_2u_2' & v_2u_2' & u_2' & u_2v_2' & v_2v_2' & v_2' & u_2 & v_2 & 1 \\ u_3u_3' & v_3u_3' & u_3' & u_3v_3' & v_3v_3' & v_3' & u_3 & v_3 & 1 \\ u_4u_4' & v_4u_4' & u_4' & u_4v_4' & v_4v_4' & v_4' & u_4 & v_4 & 1 \\ u_5u_5' & v_5u_5' & u_5' & u_5v_5' & v_5v_5' & v_5' & u_5 & v_5 & 1 \\ u_6u_6' & v_6u_6' & u_6' & u_6v_6' & v_6v_6' & v_6' & u_6 & v_6 & 1 \\ u_7u_7' & v_7u_7' & u_7' & u_7v_7' & v_7v_7' & v_7' & u_7 & v_7 & 1 \\ u_8u_8' & v_8u_8' & u_8' & u_8v_8' & v_8v_8' & v_8' & u_8 & v_8 & 1 \end{bmatrix} \begin{bmatrix} F_{11} \\ F_{12} \\ F_{13} \\ F_{21} \\ F_{22} \\ F_{23} \\ F_{31} \\ F_{32} \\ F_{33} \end{bmatrix} = 0$$

Back to Camera Calibration

- However, the calibration result is not satisfactory



Geometric Calibration

- In case of camera, we want to minimize the geometric error
  - $E = \Sigma_i^n d(x_i, x_i')$
  - Geometric error : E
  - Distance metric: d
  - Ground truth projection: x_i
  - Prediction from the homogeneous solution: $x_i' = MX_i$ (M: camera calibration matrix)
- If we know the correspondence, we can optimize the error using the camera matrix parameters.